

#09 Breadth First Search

김성수

4thrunyul@gmail.com

0. 목차



1. Breadth First Search

- Graph
- Grid Graph
- State Space Tree

1. Breadth First Search

1. Breadth First Search

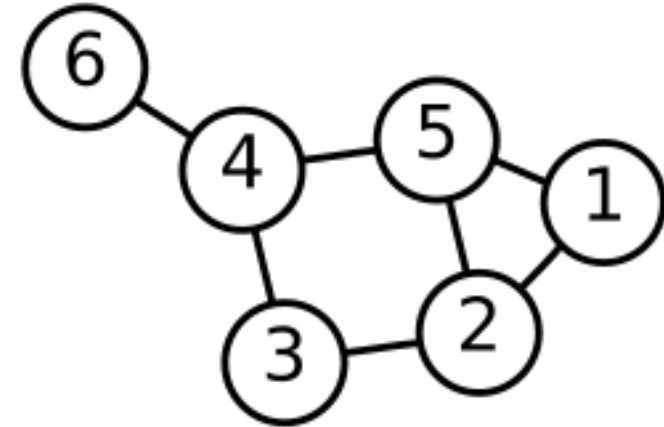
Breadth First Search(BFS, 너비 우선 탐색)

- 가능한 모든 경우를 빠짐 없이 탐색하는 Brute Force(완전 탐색) 알고리즘의 한 종류
- 현재 위치에서 갈 수 있는 모든 인접 노드를 먼저 방문한 뒤 다음 깊이로 확장하는 방식
- 현재 노드에서 같은 거리(깊이)의 모든 노드를 탐색 완료하고, 그 노드들의 인접 노드로 넘어가서 탐색 진행
- 선입선출 구조이며 주로 Queue와 반복문을 활용해 구현
- 가중치가 없는 그래프의 최단 거리 탐색, 영역 분할, 연결 요소 확인 등에 주로 활용

1. Breadth First Search: Graph

Graph Basics

- Vertex / Node(정점 / 노드): 위치나 데이터를 나타내는 단위
 - $V = \{1, 2, 3, 4, 5, 6\}$
- Edge(간선): 정점과 정점 사이를 잇는 연결선
 - $E = \{(1, 2), (1, 5), (2, 3), (2, 5), (3, 4), (4, 5), (4, 6)\}$
- Adjacent / Neighbor(인접 정점): 간선으로 직접 연결된 정점
 - 정점 4의 인접 정점은 3, 5, 6
- Degree(차수): 한 정점에 연결된 간선의 수
 - 정점 5의 차수는 3, 정점 6의 차수는 1
- Path(경로): 간선을 따라 정점을 순서대로 이동하는 경로
 - ex: $6 \rightarrow 4 \rightarrow 5 \rightarrow 1$



1. Breadth First Search: Graph



Breadth First Search: Graph

- 그래프의 한 정점에서 출발하여 갈 수 있는 가장 깊은 곳까지 탐색
- 현재 정점과 연결된 인접 정점 중 아직 방문하지 않은 곳으로 재귀 호출 진행
- 더 이상 갈 수 있는 정점이 없으면 직전 정점으로 복귀

1. Breadth First Search: Graph



Breadth First Search: Graph

- `vector<int> adj[101]` // 인접 리스트
 - `vector<int>` 101개로 구성된 1차원 배열
 - 각 정점 번호(인덱스)마다 연결된 인접 정점 목록을 동적으로 저장
- `bool visit[101]` // 방문 노드 체크 배열
 - 각 정점의 방문 여부 기록
 - 이미 방문한 정점의 재방문을 차단하여 무한 루프 방지
- `adj[u].push_back(v), adj[v].push_back(u)`
 - 양방향 그래프 표현을 위해 $u \rightarrow v, v \rightarrow u$ 양방향 경로 모두 등록
- `sort(adj[i].begin(), adj[i].end())`
 - 한 정점에서 이동 가능한 다음 정점이 여러 개일 때 오름차순으로 정렬
 - 번호가 작은 정점부터 먼저 방문하도록 탐색 순서 보장
 - 꼭 필요한 로직은 아니고 문제마다 다름

1. Breadth First Search: Graph



Breadth First Search: Graph

- 1. 시작 정점 초기화
 - 탐색 시작 노드를 큐에 삽입하고 즉시 방문 완료 표시
 - `q.push(start);`
 - `visit[start] = true;`
- 2. 큐에서 기준 정점 추출
 - 큐가 빌 때까지 선입선출 방식으로 가장 앞의 정점을 꺼내 현재 탐색 노드로 지정
 - `while(!q.empty()) {`
 - `int cur = q.front();`
 - `q.pop();`

1. Breadth First Search: Graph



Breadth First Search: Graph

- 3. 인접 정점 순회
 - 현재 정점과 연결된 정점 목록(adj[cur])을 순차적으로 확인
 - for(int next : adj[cur])
- 4. 방문 처리 및 큐 삽입
 - 아직 방문하지 않은 인접 정점을 발견 시 방문 처리 후 큐에 삽입해 다음 탐색 후보로 등록
 - if(!visit[next]) {
 - visit[next] = true;
 - q.push(next);
 - }

1. Breadth First Search: Graph



문제

- [1818 바이러스](#)
- [코드](#)

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <queue>
5
6  using namespace std;
7
8  vector<int> adj[101];
9  bool visit[101];
```

```
11 void bfs(int start) {
12     queue<int> q;
13
14     q.push(start);
15
16     visit[start] = true;
17
18     while(!q.empty()) {
19         int cur = q.front();
20         q.pop();
21
22         for(int next : adj[cur]) {
23             if(!visit[next]) {
24                 visit[next] = true;
25                 q.push(next);
26             }
27         }
28     }
29 }
```

```
31 int main() {
32     ios_base::sync_with_stdio(false);
33     cin.tie(NULL);
34
35     int n, m, u, v;
36     int cnt = 0;
37
38     cin >> n >> m;
39
40     for(int i = 1; i <= m; i++) {
41         cin >> u >> v;
42
43         adj[u].push_back(v);
44         adj[v].push_back(u);
45     }
```

```
47     for(int i = 1; i <= n; i++) {
48         sort(adj[i].begin(), adj[i].end());
49     }
50
51     bfs(1);
52
53     for(int i = 2; i <= n; i++) {
54         if(visit[i]) ++cnt;
55     }
56
57     cout << cnt;
58
59     return 0;
60 }
```

1. Breadth First Search: Graph

문제

- [1818 바이러스](#)
 - [코드](#)

정렬된 인접리스트

adj[1] : [2, 5]
adj[2] : [1, 3, 5]
adj[3] : [2]
adj[4] : [7]
adj[5] : [1, 2, 6]
adj[6] : [5]
adj[7] : [4]

1. bfs(1) 실행

- queue에 1 push
- 1번 노드 방문 처리 (visit[1] = true)
- q.pop : 1
- adj[1]를 참고하여 queue에 2와 5를 push하고 방문 처리 (visit = true)

2. 2번 노드 방문

- q.pop : 2
- adj[2]를 참고하여 queue에 3 push하고 방문 처리
- 이때 5번 노드는 1번 노드에서 방문 처리되었기에 queue에 넣지 않음

3. 5번 노드 방문

- q.pop : 5
- adj[5]를 참고하여 queue에 6 push하고 방문 처리

4. 3번 노드 방문

- q.pop : 3
- adj[3]을 참고하였으나 이미 방문한 노드 밖에 없음

5. 6번 노드 방문

- q.pop : 6
- adj[6]을 참고하였으나 이미 방문한 노드 밖에 없음
- queue가 비었기 때문에 탐색 종료

1. Breadth First Search: Graph



실습

- [5913 친구의 친구는 친구](#)
 - [코드](#)
- [2261 경로 찾기](#)
 - [코드](#)

1. Breadth First Search: Graph



5913 친구의 친구는 친구 : 아이디어

- 1. 방문 배열 정의와 해석
 - visit[] 배열을 int로 선언하여 방문 여부와 관계를 동시 관리
 - visit[1] == 1 : 1번 학생 본인(시작점)
 - visit[i] == 0 : 아직 방문하지 않은 노드
 - visit[i] == 2 : 1번 학생의 친구
 - visit[i] == 3 : 1번 학생의 친구의 친구

1. Breadth First Search: Graph



5913 친구의 친구는 친구 : 아이디어

- 2. 알고리즘 흐름
 - A. 시작점 초기화
 - `q.push(1);`
 - `visit[1] = 1;`
 - B. BFS
 - 큐에서 현재 학생의 번호(`cur`)을 꺼냄
 - 인접 리스트에 저장된 친구(`next`) 중 아직 방문하지 않은 친구(`visit[next] == 0`) 확인
 - `visit[next] = visit[cur] + 1;` 기록 후 큐 삽입 `q.push(next);`
 - C. 정답 계산
 - 2번 학생부터 N번 학생까지 순회하여 `visit[i] == 2 || visit[i] == 3`인 학생 수 카운트

1. Breadth First Search: Graph

2261 경로 찾기 : 아이디어

- 1. 간선 만들기
 - 해당 문제에서는 앞선 문제와 다르게 정점만 주어지고, 간선이 주어지지 않음.
 - 즉, 입력 받은 정점들로 직접 간선을 만들어 인접리스트(adj)에 저장 해야함
 - 모든 입력값을 2중 반복문으로 비교해 해밍 거리가 1인 정점끼리 인접리스트 생성
- 2. 부모 배열 정의와 해석
 - visit 배열을 따로 두지 않고 par[] 배열 하나로 방문 여부와 이동 경로(부모 노드)를 동시에 관리
 - $par[i] == 0$: 아직 방문하지 않은 상태 (기본값)
 - $par[s] = s$: 시작 정점 방문 표시
 - $par[next] = cur$: 직전 방문 정점을 저장하여 경로 역추적에 사용

1. Breadth First Search: Graph



2261 경로 찾기 : 아이디어

- 3. 알고리즘 흐름
 - A. 간선 구성
 - 모든 코드 쌍 (i, j) 에 대해 비트를 비교
 - 서로 다른 비트가 정확히 1개이면 $adj[i]$ 와 $adj[j]$ 에 추가
 - B. BFS
 - Queue에 시작점 s 를 삽입하고 $par[s] = s$ 기록
 - Queue에서 꺼낸 정점이 목표점 e 이면 탐색 즉시 중단 ($cur == e$)
 - 인접 정점($next$) 중 미방문 정점($par[next] == 0$)에 대해 $par[next] = cur$ 저장 후 Queue에 삽입
 - C. 경로 역추적
 - 도달 불가 : $par[e] == 0$ 이면 경로가 존재하지 않으므로 -1 출력
 - 도달 성공
 - 목표 정점 e 부터 시작하여 $cur = par[cur]$ 로 부모를 타고 올라가 시작점 s 까지 벡터에 저장
 - reverse 함수를 통해 순서를 뒤집은 뒤, 순서대로 출력

1. Breadth First Search: Grid Graph



Breadth First Search: Grid Graph

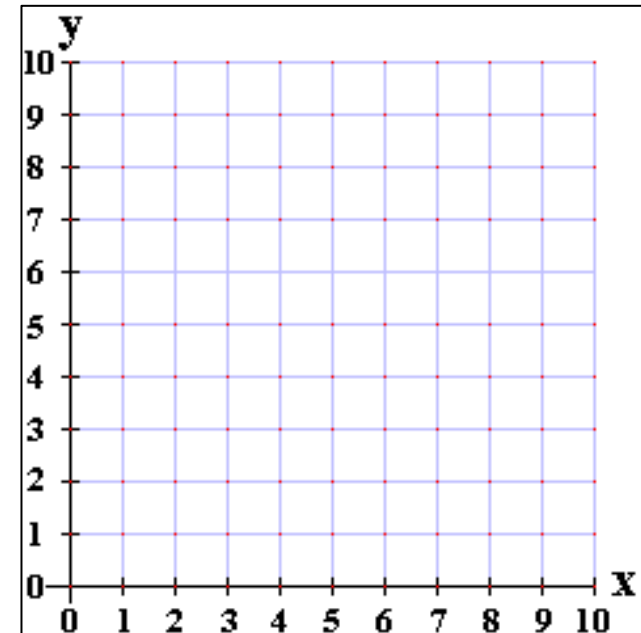
- 2차원 격자의 각 칸을 하나의 정점으로 정의 (행·열(r, c) / 좌표(x, y))
- 상하좌우 이동 처리를 위해 방향 배열(dr, dc / dx, dy) 활용
- 격자 범위(인덱스 초과), 장애물(벽), 방문 여부를 순서대로 검사하고 Queue를 활용한 너비 탐색

1. Breadth First Search: Grid Graph

Grid Graph에서 주의점

- 배열 인덱스와 수학적 좌표를 헷갈리면 안 됨
- 행·열 방식에서는 row(행)은 세로, column(열)은 가로로 정의되기에 $\text{grid}[r][c]$ 으로 바로 매핑 가능
- 좌표 방식에서는 x 를 가로, y 를 세로로 정의하기에 $\text{grid}[y][x]$ 로 매핑하고 접근해야 함
- 하지만 본인이 일관적으로 x 를 행, y 를 열로 규정하고 모든 로직에서 일관성을 유지하면 $\text{grid}[x][y]$ 사용 가능

	Column →				
Row ↓	(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
	(1,0)	(1,1)	(1,2)	(1,3)	(1,4)
	(2,0)	(2,1)	(2,2)	(2,3)	(2,4)
	(3,0)	(3,1)	(3,2)	(3,3)	(3,4)
	(4,0)	(4,1)	(4,2)	(4,3)	(4,4)



1. Breadth First Search: Grid Graph



Breadth First Search: Grid Graph

- `grid[n][m]` // 2차원 격자 맵
 - 각 격자 칸의 상태 정보(지형, 벽, 가중치, 색상 등)를 저장
 - 자료형을 bool로 둘지 int로 둘지는 문제에 따라 다름
- `bool visit[n][m]` // 2차원 방문 체크 배열
 - 각 칸의 방문 여부 기록 및 재방문 차단
- `dr[] = {-1, 1, 0, 0}`, `dc[] = {0, 0, -1, 1}` // 행·열 방향 배열
 - 현재 위치 기준 상·하·좌·우 인접 칸으로 이동하기 위한 행과 열의 변화량 정의
 - 반복문 하나로 4방향 이동을 간결하게 처리

1. Breadth First Search: Grid Graph



Breadth First Search: Grid Graph

- 1. 시작 칸 초기화
 - 탐색 시작 위치(r, c)를 큐에 삽입하고 즉시 방문 완료 표시
 - `q.push({r, c});`
 - `visit[r][c] = true;`
- 2. 인접 칸 순회(방향 탐색)
 - 현재 칸(r, c)을 기준으로 방향 배열을 이용해 상하좌우 인접 칸 좌표 계산
 - `for(int i = 0; i < 4; i++) {`
 - `int nr = q.front().first + dr[i];`
 - `int nc = q.front().second + dc[i];`
 - `}`
 - `* dr[] = {-1, 1, 0, 0}`
 - `* dc[] = {0, 0, -1, 1}`
 - `// 상, 하, 좌, 우`

1. Breadth First Search: Grid Graph



Breadth First Search: Grid Graph

- 3. 유효성 검사 및 너비 우선 탐색
 - 격자 범위 내, 이동 가능, 미방문 조건을 모두 만족하면 큐에 삽입
 - 방문 처리 시점은 문제마다 요구하는 로직에 따라 다름
 - `if(0 <= nr && nr < n && 0 <= nc && nc < m) { // 격자 범위 내인지 체크`
 - `if(grid[nr][nc] == 1 && !visit[nr][nc]) { // 이동 가능하고 미방문인지 체크`
 - `visit[nr][nc] = true; // 큐에 넣을 때 방문 처리`
 - `q.push({nr, nc});`
 - `}`
 - `}`
- 4. 현재 칸 제거
 - 현재 칸(r, c)에 대한 4방향 탐색이 모두 끝나면 큐에서 제거
 - 이후 큐에 남아있는 다음 칸을 기준으로 동일한 탐색 과정 반복
 - `q.pop();`

1. Breadth First Search: Grid Graph



문제

- [1154 빨강과 검정](#)
- [코드](#)

```
1  #include <iostream>
2  #include <string>
3  #include <queue>
4
5  using namespace std;
6
7  int w, h;
8  int cnt = 0;
9  bool board[21][21];
10 bool visit[21][21];
11 int dr[] = {-1, 1, 0, 0};
12 int dc[] = {0, 0, -1, 1};
```

```
14 void bfs(int r, int c) {
15     queue<pair<int, int>> q;
16
17     q.push({r, c});
18     visit[r][c] = true;
19     ++cnt;
20
21     while(!q.empty()) {
22         int cur_r = q.front().first;
23         int cur_c = q.front().second;
24
25         for(int i = 0; i < 4; ++i) {
26             int nr = cur_r + dr[i];
27             int nc = cur_c + dc[i];
28
29             if(nr >= 1 && nr <= h && nc >= 1 && nc <= w) {
30                 if(board[nr][nc] == 1 && visit[nr][nc] == false) {
31                     visit[nr][nc] = true;
32                     ++cnt;
33                     q.push({nr, nc});
34                 }
35             }
36         }
37
38         q.pop();
39     }
40 }
```

```
42 int main() {
43     int r, c;
44     string s;
45
46     cin >> w >> h;
47
48     for(int i = 1; i <= h; ++i) {
49         cin >> s;
50
51         for(int j = 1; j <= w; ++j) {
52             if(s[j - 1] == '.') board[i][j] = 1;
53             else if(s[j - 1] == '#') board[i][j] = 0;
54             else if(s[j - 1] == '@') {
55                 board[i][j] = 1;
56                 r = i;
57                 c = j;
58             }
59         }
60     }
61
62     bfs(r, c);
63
64     cout << cnt;
65
66     return 0;
67 }
```

1. Breadth First Search: Grid Graph



실습

- [4189 장기 2](#)
 - [코드](#)
- [1695 단지번호붙이기](#)
 - [코드](#)
- [1462 보물섬](#)

1. Breadth First Search: Grid Graph



4189 경로 찾기 : 아이디어

- 1. 이동 경로
 - 일반적인 격자 탐색에서는 상, 하, 좌, 우 4가지 이동 밖에 없었지만, 말의 경로는 8가지나 있음.
 - 즉, `dr[]`, `dc[]`를 다음과 같이 선언해야 함.
 - `int dr[] = {1, 1, -1, -1, 2, -2, 2, -2};`
 - `int dc[] = {2, -2, 2, -2, 1, 1, -1, -1};`

1. Breadth First Search: State Space Tree



Breadth First Search: State Space Tree

- 문제의 '현재 상태(좌표, 조건 플래그)'를 노드로, '선택 가능한 행동'을 간선으로 정의
- 상태(구조체 · pair)를 Queue에 담아 가까운 거리(레벨)부터 차례대로 확장
- 큐 삽입 시점에 즉시 방문 체크(visited)를 수행하여 중복 삽입 및 메모리 폭발 방지
- 간선 비용이 동일한 상태 공간에서 목표 상태에 도달하는 최단 거리·최소 횟수 보장 탐색

1. Breadth First Search: State Space Tree



문제

- [3640 잡기놀이](#)
- [코드](#)

```
1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  int n, k;
7  int visit[100001];
8
9  void bfs(int start) {
10     queue<int> q;
11
12     q.push(start);
13     visit[start] = 1;
14
15     while(!q.empty()) {
16         int cur = q.front();
17
18         if(cur == k) break;
19
20         int next_pos[3] = {cur - 1, cur + 1, cur * 2};
21
22         for(int i = 0; i < 3; ++i) {
23             int next = next_pos[i];
24
25             if(next >= 0 && next <= 100000) {
26                 if(visit[next] == 0) {
27                     visit[next] = visit[cur] + 1;
28                     q.push(next);
29                 }
30             }
31         }
32
33         q.pop();
34     }
35 }
```

```
37  int main() {
38      ios_base::sync_with_stdio(false);
39      cin.tie(NULL);
40
41      cin >> n >> k;
42
43      bfs(n);
44
45      cout << visit[k] - 1;
46
47      return 0;
48 }
```

1. Breadth First Search: State Space Tree



3640 잡기놀이 : 아이디어

- 1. 방문 배열 정의와 해석
 - visit[] 배열을 int로 선언하여 방문 여부와 해당 위치까지 걸린 시간(초)을 동시에 관리
 - visit[i] == 0: 아직 방문하지 않은 위치
 - visit[n] = 1: 시작 위치 도훈이의 좌표 (0초를 1로 표기하여 미방문 상태와 구분)
 - 시작점을 1로 초기화했으므로, 실제 최소 시간은 visit[k] - 1

1. Breadth First Search: State Space Tree



실습

- [3942 과일 대잔치](#)
- [2742 쥘노의 8퍼즐\(8puzzle-거리만\)](#)